

HelixTrack

HelixTrack

JIRA'a özgür dünyadan bir alternatif.

Özet

HelixTrack, JIRA'a (ve Belgeler eklentisi aracılığıyla Confluence'a) modern, kapsamlı ve açık kaynaklı bir alternatif sunan, çok platformlu bir proje yönetimi ve sorun takip sistemi. Go tabanlı mikro hizmetler mimarisine dayanan bu sistem, web, masaüstü ve mobil için yerel istemcilerle çalışıyor.

Kısa açıklama

Açık kaynaklı JIRA/Confluence alternatifi. Go mikro hizmetler arka ucu ("HelixTrack Çekirdeği"), proje ve sorun takibi için birleşik bir REST API arayüzü ile Confluence tarzı bir belge çalışma alanı sunuyor. Bu arayüz, HTTP/3 QUIC üzerinden web, masaüstü, Android ve iOS için yerel istemcilere servis ediliyor.

Uzun açıklama

HelixTrack, mühendislik ekiplerinin en çok bağımlı olduğu iki ürün olan JIRA ve Confluence'a açık kaynaklı, özgür bir alternatif olarak konumlandırılmış bir proje yönetimi ve sorun takip platformu. Bu iki ürünün tam bir yerine geçen HelixTrack, sahibi olduğunuz ve istediğiniz yerde çalıştırabileceğiniz bir yazılım olarak yeniden tasarlandı. Kalbinde **HelixTrack Çekirdeği** yer alıyor: Go ile Gin çatısı kullanılarak yazılmış, REST API tabanlı bir mikro hizmet. Sorun takibi, çevik/scrum panoları, ekip yönetimi ve hiyerarşik izin motoru sunan bu çekirdek, yerel bir işlem içi motor ile HTTP destekli bir hizmet arasında değiştirilebilir bir yetkilendirme modeline sahip. Böylece aynı yetkilendirme sistemi, tek bir dizüstü bilgisayardan dağıtık bir kümeye kadar her ölçekte uygulama koduna dokunmadan çalışabiliyor. REST arayüzünü onlarca farklı rotaya yaymak yerine, Çekirdek tüm işlemleri tek

bir eylem yönlendirmeli /do uç noktası üzerinden yürütüyor. İstek/yanıt yapısı da tek tip (action/jwt/object/data giriş, errorCode/errorMessage/data çıkış). Her istemci aynı küçük sözleşmeyi kullanıyor ve yeni bir özellik eklemek, yeni bir URL tanımlamak yerine yalnızca bir eylem eklemek anlamına geliyor. Çekirdek, HTTP/3 QUIC üzerinden iletişim kuran, ayrık Kimlik Doğrulama, İzin Yönetimi ve Yerelleştirme hizmetleriyle entegre çalışıyor. Bu hizmetler ayrı makinelerde veya kümelerde çalıştırılabilir ya da test ortamlarında tamamen devre dışı bırakılabilir. Veriler, sıfır kurulumlu geliştirme için SQLite’de, üretim ortamında ise PostgreSQL’de saklanıyor. Ayrıca SQLCipher (AES-256) ile disk üzerinde şifreleniyor; böylece hassas proje verileri varsayılan olarak korunuyor, ek bir önlem olarak değil. **Belgeler V2** eklentisi, takip sistemini tam bir bilgi platformuna dönüştürüyor: Confluence tarzı bir çalışma alanı sunan bu eklenti, alanlar, sayfalar, sürüm kontrolü, şablonlar, gerçek zamanlı WebSocket iş birliği ve analizler içeriyor. Böylece wiki ve sorun takip sistemi nihayet tek bir arka uç altında birleşiyor, iki ayrı ürünün bir araya getirilmesiyle değil. Çekirdeğin etrafında ise birden fazla istemci uygulaması bulunuyor: Angular web istemcisi, Tauri + Angular masaüstü istemcisi, yerel Android (Kotlin) ve iOS (Swift) uygulamaları, ayrıca HarmonyOS ve Aurora OS istemcileri ile bir ekran koruyucu. Tüm bu istemciler aynı arka uca bağlanıyor ve UDP yayını sayesinde yerel ağlarda sunucuyu otomatik olarak keşfediyor. Böylece yeni bir istemci, elle yapılandırma gerektirmeden sunucusunu bulabilir. İstemci uygulamaları ayrı, özel depolarda tutuluyor ve burada yalnızca ürün düzeyinde sunuluyor.

Neden inşa ettik

Ekiplere, JIRA + Confluence yığınının gerçekten açık, kendi sunucularında barındırılabilir bir alternatif sunmak için — “özgür dünyaya” yönelik olarak — satıcı bağımlılığı olmadan, kurumsal düzeyde izleme, belgeler ve iş birliğini tek bir açık kaynak lisansı altında birleştirdik.

Neden oyunun kurallarını değiştiriyor

İki ağır ticari ürünü — sorun izleme ve wiki/belge yığını — tek bir açık, yüksek performanslı, kendi sunucunuzda barındırılabilir platformda birleştiriyor ve bunu mevcut rakiplerin hiç sunmadığı bir şeyle eşleştiriyor: Gerçek çok platformlu *yerel* istemciler (web, masaüstü, Android, iOS, ayrıca HarmonyOS ve Aurora OS). Hepsi tek bir arka uç sözleşmesiyle çalışıyor. Kazanç, ödün vermeden sahiplik. HTTP/3 her yerde, tamamen ayrıştırılmış mikro hizmet mimarisi ve beklemedeki veriler için SQLCipher AES-256 şifrelemesi, genellikle yalnızca tescilli SaaS sistemlerine özgü performans ve güvenlik seviyesini, kendi altyapınızda

barındırdığınız bir sisteme taşıyor — koltuk lisansları yok, satıcı bağımlılığı yok, verileriniz altyapınızdan çıkmıyor. Ekipler, JIRA-plus-Confluence deneyimini zaten bildikleri şekilde, kendi donanımlarında, tek bir açık kaynak lisansı altında yaşıyor.

Yenilikçi yönleri

- Eylem tabanlı birleşik /do API — tek bir uç nokta, tek bir zarf, eyleme yönlendirme. Yeni yetenekler yeni URL'ler yerine yeni eylemler olarak geliyor; böylece saldırı yüzeyi, istemci kodu ve dokümantasyon yükü, tüm platformların paylaştığı tek bir sözleşmeye indirgeniyor.
- HTTP/3 QUIC, varsayılan hizmetler arası iletişim aracı olarak — modern, düşük gecikmeli, bağlantı dayanıklı ağ iletişimi, sonradan eklenmiş değil, baştan itibaren entegre.
- Yerel, süreç içi bir uygulama ile HTTP destekli bir hizmet arasında değiştirilebilir bir izin motoru; isteğe bağlı, bağımsız olarak dağıtılabilir Kimlik Doğrulama, İzinler ve Yerelleştirme hizmetleriyle birlikte — tek bir süreçte ya da kümede çalıştırsanız da aynı yetkilendirme modeli.
- --space-root bayrağıyla çoklu alan veri yalıtımı: Her proje kendi izole veritabanına ve varlık deposuna sahip oluyor; böylece kiracılar ve projeler, sorgu filtreleriyle değil, depolama sınırında ayrılıyor.
- Beklemedeki veriler için SQLCipher AES-256 şifreleme — hassas proje verileri, varsayılan olarak ve şeffaf bir şekilde diskte korunuyor.
- Yerel ağlarda UDP yayınıyla otomatik istemci-sunucu keşfi — istemci, Core'u sıfır manuel yapılandırma ile buluyor.
- Belgeler V2, gerçek bir "Confluence alternatifi" — iyimser kilitlemeyle paralel düzenleme, çakışma tespiti ve tam değişiklik geçmişi sunan, izleyiciyle aynı arka uca sahip gerçek iş birliği belgeleri.

En büyük teknik zorluklar ve çözümlerimiz

- **Altı istemci platformu, tek arka uç, sıfır sözleşme sapması.** Web/Angular, Masaüstü/Tauri, Android/Kotlin, iOS/Swift, HarmonyOS ve Aurora istemcilerini sürdürmek, normalde altı farklı API entegrasyonu anlamına gelir ve bunlar zamanla senkronizasyonu kaybeder. Bu riski, tek eyleme yönlendirmeli /do API ve sabit zarfını *tek* sözleşme haline getirerek ortadan kaldırdık — her istemci bunu aynı şekilde hedefliyor. Üstüne UDP yayınıyla hizmet keşfini ekledik; böylece istemciler Core'u ağda elle yapılandırılmış uç noktalar olmadan bulabiliyor.

- **Hizmetleri ayırırken gecikme bedeli ödememek.** Kimlik Doğrulama, İzinler ve Yerelleştirme'yi bağımsız olarak dağıtılabilir hizmetlere bölmek, normalde her çağrı için ek bir ağ atlama anlamına gelir. Tüm hizmetler arası çağrılarda HTTP/3 QUIC kullanarak bu atlamaları hızlı ve bağlantı dayanıklı tuttuk; her hizmetin bağımsız olarak çalıştırılabilir olmasını sağladık — hatta test yapılandırmalarında tamamen devre dışı bırakılabilir — böylece ayrıştırma, sabit bir maliyet değil, dağıtım tercihi haline geldi.
- **Confluence düzeyinde iş birliği, yazım kaybı karmaşası olmadan.** Gerçek zamanlı çok yazarlı düzenleme, çakışan yazımlara davetiye çıkarır. Belgeler V2, iyimser kilitleme altında alanlar/sayfalar/sürümler, açık çakışma tespiti, geri dönülebilir tam değişiklik geçmişi ve gerçek zamanlı WebSocket senkronizasyonu bu çözünüyor — iş birliği tutarlı kalıyor, düzenlemeler sessizce ezilmiyor.
- **Beklemedeki şifreleme, performansı öldürmeden.** SQLCipher AES-256 diskteki verileri korur, ancak her sorgu için ek yük getirir. Bunu, çok katmanlı önbelleklemeyle (Yerelleştirme hizmetindeki Redis önünde bellek içi LRU) dengeledik; böylece çok dilli aramalar gibi sıcak yollar hızlı kalırken veriler şifreli duruyor.

Teknoloji Yığını

- **Go + Gin** — yüksek iş hacmi ve düşük gecikme süreli HTTP hizmetleri için tek ikili dosya dağıtım modeliyle seçildi; Core'un REST API bileşenini, JWT/CORS ara yazılımını ve tüm sistemi yönlendiren eylem tabanlı /do yönlendiricisini içerir.
- **HTTP/3 QUIC** — Core ile Kimlik Doğrulama/Yetkilendirme/ Yerelleştirme hizmetleri arasındaki iletişim için seçildi; QUIC'un çoklu bağlantı ve bağlantı aktarım tasarımı, TCP'un takıldığı durumlarda kuyruk gecikmesini azaltır ve dengesiz bağlantılarla başa çıkar.
- **PostgreSQL (prod) / SQLite (dev)** — hem izleme hem de belgeler şemasını kapsayan büyük ölçekli ilişkisel model için tek bir veritabanı altyapısı: SQLite, yerel geliştirmeyi sıfır kurulum ve dosya tabanlı hale getirirken, Postgres üretim ölçeğinde özel bir production compose profili aracılığıyla devreye girer.
- **SQLCipher (AES-256)** — hassas proje verilerinin korunması için uygulama katmanında ek şifreleme gerektirmeyen, sorgu yazımını değiştirmeyen ve veritabanı düzeyinde şeffaf şifreleme sağlayan çözüm olarak seçildi.
- **Redis** — Yerelleştirme hizmetinin arkasında bellek içi LRU önbelleği ile birlikte kullanılan paylaşımlı önbellek katmanı olarak seçildi; bu iki katmanlı önbellek, altında şifreleme yükü olsa bile sıcak çok dilli sorguları hızlı tutar.
- **Uber Zap + Lumberjack** — yapılandırılmış, düşük bellek kullanımlı ve otomatik log döndürme özelliğine sahip

günlükleme sistemi olarak seçildi; böylece Core, sınırsız log büyümesi olmadan üretim ortamında gözlemlenebilir kalır.

- **golang-jwt / JWT** — durum bilgisi tutmayan kimlik doğrulama mekanizması olarak seçildi; imzalı token, her /do zarfının jwt alanında taşınır ve böylece tüm istemcilerde kimlik doğrulama tek tip hale gelir.
- **Angular 19 (+ Material, RxJS)** — tepki veren, bileşen odaklı bir tarayıcı istemcisi için, kutudan çıkan olgun Material tasarım sistemiyle birlikte seçildi.
- **Tauri 2.0 + Rust + Angular** — Angular arayüzünü Rust destekli bir web görünümü içinde yeniden kullanarak tam bir tarayıcı çalışma zamanı paketlemek yerine, küçük bir ayak iziyle yerel bir masaüstü kabuğu sunmak için seçildi.
- **Kotlin (Android) / Swift + SwiftUI (iOS)** — mobil kullanıcıların sarılmış bir web görünümü yerine gerçekten yerel, platforma özgü istemcilere sahip olmasını sağlamak için seçildi.
- **Docker / Docker Compose (Podman uyumlu)** — yeniden üretilebilir, konteyner tabanlı dağıtım için seçildi; /health kontrolleri entegre edilmiş durumda ve Podman uyumluluğu sayesinde herhangi bir arka plan hizmeti veya satıcı zorunluluğu bulunmuyor.
- **Testify (Go); Cypress/Playwright/Karma+Jasmine (istemciler)** — tek bir arka uç ve çok sayıda istemci mimarisine uygun olarak, arka uç sözleşmesini ve istemci arayüzlerini bağımsız olarak kapsayan katmanlı otomatik testler için seçildi.

Durum ve Dürüstlük Notları

- **Durum: beta.** HelixTrack Core, çalışan bir REST API mikro hizmetidir; **Belgeler V2** eklentisi yaklaşık %95 tamamlanmış olarak belgelenmiş olup, bilinen bir veritabanı alan eşleştirme sorunu nedeniyle henüz tam olarak sunulmamaktadır.
- **Lisans: Belirlenmedi.** CLAUDE.md dosyasında MIT lisansı belirtilmiş olsa da, kayıtlı core/LICENSE dosyası Apache 2.0'dır — lisansın kesin olarak belirlenebilmesi için bu çelişkinin giderilmesi gerekmektedir.
- Projenin README dosyasında belirtilen performans rakamları (örn. 50.000+ istek/saniye, milisaniyenin altında sorgu süreleri) bağımsız olarak yayımlanmış ölçümler değil, tasarım/hedef değerleridir ve bu nedenle yukarıdaki iddialarda yer almamıştır.
- İstemci uygulamaları (Web, Masaüstü, Android, iOS, Aurora, HarmonyOS) **özel** depolarda bulunmaktadır ve yalnızca ürün düzeyinde tanımlanmıştır.

İçerik

Öncelik kademesi: Helix-birincil ve Helix-Track ürün serisinin amiral gemisi — Server Factory projelerinin önünde yer alır.